Enhancement One: Software Design and Engineering

Artifact Description

The artifact I selected for this enhancement is the Travlr Getaways full-stack web application that I originally developed in CS-465 Full Stack Development. The project uses MongoDB, Express, Angular, and Node.js to provide both a customer-facing travel website and an administrative portal for managing trips and reservations.

I chose this project because it is the largest and most technically complex application I created during the program. It combines frontend development, backend services, database management, authentication, authorization, and administrative functionality within a single system. Because the application already contained multiple layers and user roles, it provided a strong foundation for demonstrating software design and engineering improvements.

Enhancement Description

For this enhancement, I focused on improving the application's architecture, maintainability, and security while preserving its existing functionality. Although the original application successfully met its functional requirements, several areas of the backend could be improved through refactoring and better separation of responsibilities.

One of the most significant improvements was the creation of a dedicated authentication middleware. In the original implementation, authentication logic was embedded directly within route definitions, resulting in repeated code and making the authentication process more difficult to maintain. To address this issue, I created an authenticateJWT middleware component that verifies JSON Web Tokens (JWTs), validates user identity, and attaches authenticated user information to the request object. This middleware is now executed before protected routes are processed, allowing controllers to focus solely on business logic rather than authentication responsibilities.

I also implemented a role-based authorization middleware to control access to administrative functionality. The original application performed permission checks within individual controller methods, which created duplicated logic and increased maintenance complexity. The new authorization middleware evaluates the authenticated user's role before allowing access to protected resources. By centralizing access control, administrative permissions can be modified in a single location without requiring changes across multiple controllers or routes. This design follows the principle of separation of concerns and improves long-term maintainability.

Another enhancement involved standardizing API responses throughout the backend. Previously, controllers returned error messages using different formats, making debugging and frontend integration more difficult. To address this issue, I created a reusable API response utility that provides a consistent structure for error handling across the application. Controllers now use a shared sendError function that returns standardized status codes and error messages. This approach improves communication between the frontend and backend while making application behavior more predictable and easier to troubleshoot.

In addition to these architectural improvements, I enhanced documentation throughout the project. Descriptive file headers, comments, and section labels were added to controllers, middleware, routes, models, and configuration files. These additions improve readability and make it easier for future developers to understand how the different components interact within the application.

Alignment to Course Outcomes

This enhancement supports Course Outcome 3 because it focuses on improving the design and implementation of an existing software solution using accepted software engineering principles. By moving authentication, authorization, and API response handling into reusable middleware and utility components, I improved modularity, reduced code duplication, and increased maintainability. The resulting architecture is more scalable and better aligned with professional software development practices.

This enhancement also supports Course Outcome 5 by strengthening the security architecture of the application. Authentication is now handled through a centralized JWT middleware, while authorization is enforced through dedicated role-based access controls. These changes help ensure that protected resources can only be accessed by authorized users and reduce the risk of inconsistent security checks across the application. Combined with secure password hashing and token-based authentication, these improvements contribute to a more secure overall system.

Progress Toward Planned Outcomes

My original goals were to improve application organization, strengthen authentication and authorization, standardize error handling, and increase maintainability. Through the implementation of reusable middleware components, centralized authorization management, and standardized API responses, I successfully achieved these objectives.

The completed enhancements closely align with the original enhancement plan. Authentication and authorization responsibilities were separated from controllers and routes, reducing duplicated code and improving modularity. Error handling was standardized through a reusable

utility, improving consistency across the application. Documentation improvements further enhanced maintainability by making the codebase easier to understand and navigate.

As a result, I do not anticipate making significant changes to my outcome-coverage plan because the completed enhancements effectively address the goals identified during the code review process.

Reflection

This enhancement helped me better understand the difference between developing software that simply works and developing software that can be effectively maintained and expanded over time. While the original application was functional, many responsibilities were concentrated within controllers and route definitions, making future modifications more difficult.

The most valuable improvement was the implementation of reusable authentication and authorization middleware. Separating these concerns from application logic resulted in a cleaner architecture and significantly reduced duplicated code. It also made future changes easier because security-related functionality is now managed from a centralized location rather than scattered throughout the application.

Another important lesson involved the value of standardized error handling. During testing, I discovered that inconsistent response formats made it more difficult to diagnose issues between the frontend and backend. Implementing a shared API response utility simplified debugging and provided more predictable behavior across the application.

Testing played an important role in validating these enhancements. After implementing the middleware components, I verified that protected routes correctly required authentication and that administrative routes were inaccessible to unauthorized users. I also tested CRUD operations to ensure that refactoring the authentication and authorization logic did not introduce regressions into existing functionality. Several debugging sessions were required to verify JWT validation, role assignment, and route protection behavior before the final implementation was completed successfully.

Overall, this enhancement allowed me to improve the quality of the application without changing its core functionality. The final result is a more organized, maintainable, secure, and scalable application that better reflects professional software engineering practices and the skills I have developed throughout the Computer Science program.